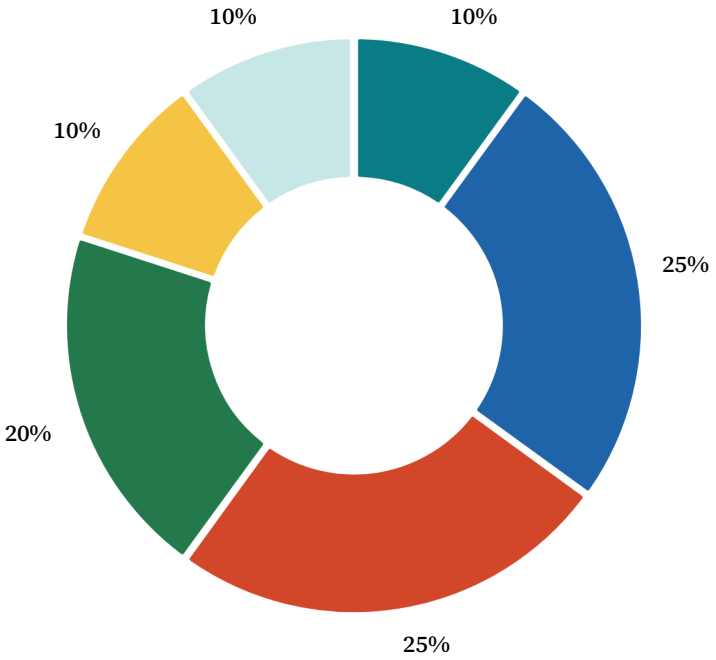
Marking Rubric for IT7006 OOD project

Prepared by:

Dr. Kelly and Dr. Faustino

Marks at a glance



SECTION	MARKS	MAIN FOCUS
Project Overview	10	Problem, objectives, users and requirements
UML Design	25	OO concepts, UML diagrams and design quality
Object Oriented Design Concepts	25	OOP Design Concepts
Design Pattern (ERD)	20	Pattern selection, justification and implementation
User Interface & System Implementation	10	Interface & System Implementation
Data Layer & Database	10	Database structure and data layer

1. Project Overview – 10 marks

Evidence expected

Students should briefly explain:

- What problem the system solves
- Purpose and objectives of the system
- Who will use the system
- Main functions/features

CRITERIA	MARKS
Problem statement	3
Project objectives	2
Target users	2
Main system functions	3
Total	**10**

2. UML Design – 25 marks

CRITERIA	MARKS	FOCUS
Use Case Diagram	4	Appropriate actors, use cases and relationships
Class Diagram	15	Classes, attributes, methods, relationships and multiplicity
Sequence Diagrams	4	Correct object interaction and messages
UML consistency	2	UML diagrams are consistent with each other

Particular marking focus for Class Diagram

The class diagram should demonstrate that students can translate requirements into an appropriate object-oriented structure.

Look for:

- Appropriate identification of classes
- Appropriate responsibilities
- Correct attributes and operations
- Appropriate visibility
- Correct multiplicity
- Association
- Aggregation
- Composition
- Inheritance
- Interface implementation
- Avoidance of unnecessary relationships

⚠ **Important:** Do not award marks simply because students have included every type of relationship. The relationship must be **appropriate to the problem domain**.

3. Object-Oriented Concepts – 25 marks

OO CONCEPT	MARKS	WHAT TO ASSESS
Encapsulation	5	Data hiding, appropriate access modifiers, properties/methods and protection of object state
Abstraction	4	Appropriate representation of essential characteristics while hiding unnecessary implementation details
Inheritance	4	Appropriate parent/child relationship and reuse/generalisation
Polymorphism	5	Same interface/base type with different implementations/behaviour
Interface	3	Appropriate interface design and implementation
OO design quality	4	Cohesion, responsibility assignment, maintainability and appropriate relationships
Total	**25**	

3.1 Encapsulation – 5 marks

5 – Excellent

- Object state is appropriately protected.
- Appropriate access modifiers are used.
- Classes control how their data is accessed or modified.
- Student can clearly explain why encapsulation is used.

3–4 – Good

- Encapsulation is generally applied correctly.
- Some minor design issues exist.

1–2 – Weak

- Some classes use encapsulation, but important data is directly exposed.
- Limited understanding demonstrated.

---

0 – Not demonstrated

- No meaningful evidence of encapsulation.

3.2 Abstraction – 4 marks

Assess whether students identify **what an object needs to expose without unnecessarily exposing implementation details**.

For example:

Payment: processPayment(), validatePayment()

rather than exposing all internal payment-processing details to the user of the class.

3.3 Inheritance – 4 marks

Assess:

- Appropriate use of generalisation
- Correct parent/child relationship
- Reuse of common attributes/behaviour
- Whether inheritance is actually justified

> ⚠ **Important marking point:** Students should not create inheritance simply to demonstrate that they know the concept.

For example: Employee, with children Manager and Lecturer, should represent a meaningful **is-a** relationship.

3.4 Polymorphism – 5 marks

Assess whether:

- A common interface/base type is used
- Different subclasses provide different behaviour
- The same method/message can produce different behaviour
- The student can explain why polymorphism is beneficial

> For example: Payment, with subclasses CreditCardPayment, PayPalPayment and BankTransferPayment. Each implementation could provide its own: processPayment()

The important point is **different behaviour through a common abstraction**, rather than simply having methods with the same name.

3.5 Interface – 3 marks

Assess:



Appropriate interface definition



Appropriate implementation by classes



Interface represents a meaningful contract



Student understands the difference between an interface and a concrete class

3.6 Overall OO Design Quality – 4 marks

Assess whether students have assigned responsibilities appropriately.

Look for:

- High cohesion
- Appropriate class responsibilities
- Low unnecessary coupling

- Meaningful relationships
- Reusable classes
- Maintainable design

4. Design Pattern – 20 marks

Students should implement **at least one design pattern taught in the course**, such as:

- Singleton
- Observer
- Builder

However, the mark should not be awarded simply for including a pattern.

CRITERIA	MARKS
Identification of design problem	4
Selection and justification of pattern	4
Correct UML implementation	4
Correct programming implementation	4
Explanation of pattern participants and interactions	2
Evaluation of benefits/limitations	2
Total	**20**

4.1 to 4.3: Problem, selection and UML

4.1 Identification of Design Problem – 4 marks

Excellent: Clearly identifies a genuine design problem in their system and explains why a pattern is useful.

Weak: Pattern is included but the underlying problem is unclear.

4.2 Pattern Selection & Justification – 4 marks

Students should explain: Why was this pattern selected for this particular problem?

For example: The Observer pattern is used because multiple components need to be notified automatically when the order status changes.

This is stronger than: We used Observer because it is a design pattern.

4.3 UML Implementation – 4 marks

Assess whether the UML design correctly represents the selected pattern.

Students should identify:

- Pattern participants
- Classes/interfaces involved
- Relationships
- Responsibilities

4.4 to 4.6: Code, explanation and consequences

4.4 Programming Implementation – 4 marks

Assess whether the actual implementation follows the selected pattern.

The code should correspond to the UML design.

4.5 Explanation – 2 marks

Students should be able to explain:

- Which classes participate
- What each class does
- How the classes interact
- Where the pattern is used

4.6 Benefits & Limitations – 2 marks

Students should discuss the consequences of using the pattern.

Benefits: reduces duplicated code; improves flexibility; separates responsibilities.

Potential limitations: adds additional classes/complexity; may be unnecessary for a very small system.

> This encourages students to understand that **design patterns are solutions to particular design problems, not features that should be added automatically**.

5. User Interface & System Implementation – 10 marks

The UI should provide evidence that the designed classes and business logic are actually being used.

CRITERIA	MARKS
UI functionality	3
UI usability/consistency	2
Integration with OO classes	3
Overall system functionality	2
Total	**10**

6. Data Layer & Database – 10 marks

Students should demonstrate that the application can communicate with the database rather than relying on hard-coded data.

CRITERIA	MARKS
Database design	3
Data-layer implementation	3
CRUD operations	2
Integration with application	2
Total	**10**

Overall Performance Descriptors

LEVEL	GENERAL DESCRIPTION
Excellent	Correct implementation, appropriate design decisions, strong justification and clear understanding demonstrated
Good	Mostly correct implementation with minor design or explanation issues
Satisfactory	Basic competency demonstrated, but some concepts are applied mechanically or inconsistently
Weak	Limited implementation or significant conceptual/design problems
Not Demonstrated	Required competency is absent or cannot be evidenced

One important marking principle

For this course, we distinguish between **"using an OO feature"** and **"demonstrating OO competency."**

❌ Student has a parent class and two child classes → automatically gets full inheritance marks.

✅ Student has a meaningful inheritance hierarchy, explains why the **is-a relationship** exists, places common responsibilities in the parent class, extends specialised behaviour in child classes, and demonstrates polymorphic use.

❌ "We used Singleton in our project." should receive limited credit.

✅ "We identified that only one database connection manager should coordinate database access. The Singleton pattern was selected for this reason. The UML shows the Singleton structure, the implementation enforces a single instance, and the student explains the advantages and limitations." would demonstrate the **design thinking** the final project is intended to assess.